School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	[redacted]
Student Name:	Maël Conejos Crusson

Module Information

Module Title:	Scripting for Cybersecurity
Module Code:	[redacted]

Assessment Information

Assessment Title:	Group Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	13/12/2025

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	[redacted]
Student Name:	Mark Diadyk

Module Information

Module Title:	Scripting for Cybersecurity
Module Code:	[redacted]

Assessment Information

Assessment Title:	Group Project

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	13.12.2025

Scripting for Cybersecurity Project

Automated Threat Intelligence Auditor

Introduction

This project focuses on automating the enrichment and analysis of suspicious network traffic on a Windows system. We chose this cybersecurity automation tool because manually checking multiple suspicious IP addresses against public threat databases (like AbuseIPDB) is time-consuming and inefficient.

We (as the project requested) used both PowerShell and Python scripts which were orchestrated by a master script to have all scripts run in one go, making the tool easy to use.

Tool Design & Implementation

Overall Structure

The tool is designed as a multi-stage pipeline where each script performs a separate set of actions. PowerShell is used for system automation and reporting. Python is used for API interaction and data analysis. A master PowerShell script orchestrates the entire workflow to ensure correct execution order.

Log Extractor

A PowerShell script the script bypasses log extraction and is hardcoded to write a list of high-risk, diverse IP addresses (hardcoded for demonstration purposes) and saves them to an intermediary file: “ip_list.txt”. Additionally, it checks whether it is running as admin or not and catches errors.

API Scanner

A Python script queries AbuseIPDB with IPs from ip_list.txt. It collects abuse confidence score, country code, and ISP information for each of the IPs. All this data is then saved to a separate csv file: “scan_results.csv”.

Stat Cruncher

A Python script processes data from scan_results.csv. It counts the number of IPs and high-risk IPs (abuse confidence score higher than 50). It calculates the country with the greatest number of occurrences. It then puts those findings in a text file: “final_summary.txt”.

Report Viewer

A PowerShell script reads final_summary.txt and prints it to the command line. It prints lines with added foreground colours: high-risk IPs and error - red, (if the count is greater than zero, indicating

active threats, or dark green if the count is zero, indicating a clean audit result), primary source country - yellow, everything else - white.

Master Run

A PowerShell script orchestrates the execution of all other scripts in order.

Cybersecurity Context

Firstly, regarding Indicators of Compromise, the original design targeted Event ID 4625 (Failed Logon) as the primary signature of brute-force and password spraying attacks. The current iteration uses a guaranteed IP list to successfully demonstrate the full functionality of the subsequent Threat Intelligence and Statistical Analysis components.

Then, regarding The Value of Intelligence, querying an API (like AbuseIPDB which we used in the project) is definitely essential. It allows analysts to tell the difference between a user mistyping a password (leaning onto low risk) and a malicious IP address from a high-risk country (which leans into the high risk).

And finally regarding the impact. The tool we created reduces the 'Mean Time to Detect' by automating the lookup process.

Tool Demonstration

The prerequisites for this project (which are outlined in the 'README.md' file) are primarily Administrator privileges, having Python installed, and the API key set.

To demonstrate the use of this tool, it was run on a Windows 11 personal desktop, running Windows PowerShell as administrator (Fig. 1).



Figure 1: Master_Run.ps1 Output

Reflection

Maël's Contribution

I worked on both Python scripts ('2_ApiScanner.py' and '3_StatusCruncher') as well as the orchestrator PowerShell script ('Master_Run.ps1').

As for the documentation, I worked on both the Introduction and the Cybersecurity Context.

Mark reviewed and helped with my work as I reviewed and helped with his work so that we were in agreement with every element of this project.

Mark's Contribution

I worked on two PowerShell scripts ('1_LogExtractor.ps1' and '4_ReportViewer.ps1') and markdown README file.

I wrote tool design & implementation and tool demonstration sections as well as completing the demonstration itself.

Challenges

The first challenge was actually creating the scripts themselves since this tool is fairly complex. Luckily, we could base ourselves on previous labs as well as some online examples (which we did not fully use, we just observed them to understand how everything works). Stack Overflow was good help.

The second we challenge we faced was actually running the orchestrator script. The issue was in the log extractor PowerShell script: you need to have Admin privileges to view failed login attempts (the 'Get-WinEvent' on line 19). And executing the script with admin privileges was quickly solved by simply running PowerShell with admin privileges. But, the second issue when trying to run the script normally: and as it turned out, simply executing this command '.\Master_Run.ps1' (when in the correct directory) was not enough. The command we ended up using that actually worked was 'powershell -File ./Master_Run.ps1'. Running this allows to run the powershell executable 'powershell' and passing the script as an argument to its '-File' parameter. This command was found on stack overflow, where a user had a similar issue (link 1). After a while however, we could execute the command normally ('.\Master_Run.ps1').

The final and most significant challenge was the inability to read the Windows Event Logs (Security and Application) even when running as Administrator on the host desktop. It repeatedly caused errors. After confirming the failure of the Get-WinEvent command in all contexts, we changed the project's direction: we eliminated the complex log extraction and implemented a guaranteed data injection method. This proved that while the environment blocked access to system logs, the core threat intelligence and analysis components of the tool are fully functional.

References

Link 1: https://stackoverflow.com/questions/2035193/how-to-run-a-powershell-script